

Sistemas Numéricos

Son fundamentales en la programación porque los ordenadores y sistemas digitales se basan en la manipulación y representación de datos en diferentes bases numéricas, principalmente el **sistema binario**. Pero es importante conocer algo de otros sistemas por las siguientes razones.

1. Representación interna de datos

Los sistemas numéricos, especialmente el **binario** (base 2), son esenciales para entender cómo los **computadores almacenan y procesan información**. En el hardware, los datos se representan con secuencias de bits (0s y 1s), lo que hace que el sistema binario sea el formato básico para todo tipo de datos: números, caracteres, imágenes, etc. Otros sistemas numéricos como el **octal** y **hexadecimal** se utilizan porque ofrecen una forma más compacta y legible para representar números binarios.

2. Optimización y control de bajo nivel

Cuando se trabaja con programación de bajo nivel o programación de sistemas (como en el desarrollo de controladores o sistemas embebidos), es crucial manipular directamente bits y bytes. Los sistemas numéricos nos permiten controlar los datos a nivel binario para optimizar el uso de la memoria, realizar operaciones bit a bit, y trabajar con instrucciones específicas del hardware.

Ejemplo: Manipulación de bits (operaciones bitwise) en lenguajes como C para configurar registros de hardware o gestionar permisos en sistemas operativos.

3. Conversiones entre sistemas numéricos

En programación, es común convertir datos entre diferentes sistemas numéricos:

- **Hexadecimal** es más legible que el binario y se utiliza frecuentemente para representar direcciones de memoria o valores de color en gráficos.

- **Decimal** es el sistema que usamos normalmente para entrada/salida de datos y cálculos, pero internamente la máquina trabaja en binario.

Saber cómo convertir entre sistemas (binario, decimal, hexadecimal) es útil para depurar programas y entender representaciones de datos en memoria.

Ejemplo: Los colores en una pantalla suelen representarse en hexadecimal. Un color RGB (rojo, verde, azul) como #FF5733 se traduce en valores numéricos que la computadora interpreta en binario.

4. Operaciones aritméticas y precisión

Conocer cómo se representan los números enteros y fraccionarios (punto fijo o flotante) ayuda a evitar errores relacionados con la Precisión numérica. Esto es clave en aplicaciones de cálculos científicos, gráficos por computadora o cualquier sistema que requiera alta precisión.

Ejemplo: Comprender la notación de punto flotante es crucial para evitar errores de precisión cuando se manejan decimales grandes o pequeños en cálculos financieros o gráficos.

5. Eficiencia y optimización del código

Conocer los sistemas numéricos también ayuda a mejorar la eficiencia del código. Por ejemplo, los programadores pueden utilizar operaciones con bits para realizar cálculos más rápidos o ahorrar memoria. Saber cuándo y cómo utilizar sistemas como el binario o hexadecimal es útil para escribir código más rápido y eficiente.

Ejemplo: Las operaciones de desplazamiento de bits (shift) son mucho más rápidas que las multiplicaciones o divisiones convencionales, y son muy útiles en situaciones donde se requiere máxima eficiencia, como el procesamiento de gráficos o criptografía.

6. Criptografía y seguridad

En aplicaciones de seguridad y criptografía, los sistemas numéricos (especialmente hexadecimal y binario) juegan un papel importante. Los algoritmos de cifrado, las claves y los hashes a menudo están representados en sistemas numéricos, y manipular estos números es parte del proceso de cifrado y descifrado.

Ejemplo: El algoritmo de cifrado AES utiliza operaciones sobre datos en formato binario para realizar transformaciones seguras de la información.

7. Depuración y análisis de errores

Durante la depuración, los programadores a menudo necesitan examinar los datos en diferentes formatos numéricos para encontrar errores. Los valores de las variables en binario o hexadecimal pueden proporcionar pistas sobre errores de desbordamiento o de cómo se está gestionando la memoria.

Ejemplo: Si un programador está trabajando con direcciones de memoria en un programa de bajo nivel, el uso del sistema hexadecimal facilita la lectura y comprensión de esas direcciones, que en binario serían muy largas y complicadas.

Conclusión:

Comprender los **sistemas numéricos** es esencial para los programadores porque proporciona las bases necesarias para:

- Manipular datos de manera eficiente.
- Optimizar el código.
- Entender cómo funcionan los sistemas digitales.
- Depurar programas y corregir errores.
- Trabajar con precisión en áreas como la criptografía, gráficos por computadora y cálculo científico.

Tener un conocimiento sólido de cómo los diferentes sistemas numéricos interactúan con el hardware y el software es clave para escribir programas robustos y eficientes.

Sistemas numéricos

Los sistemas numéricos son formas de representar cantidades utilizando diferentes bases.

En la computación y las matemáticas, los más utilizados son el sistema binario, el sistema decimal, el sistema octal y el sistema hexadecimal.

1. Sistemas numéricos

Los sistemas numéricos se definen por **una base** o **radix**, que es el número de símbolos diferentes que se pueden utilizar para representar valores en ese sistema. Por ejemplo:

- **Sistema decimal** (base 10): Usa 10 símbolos (0-9).
- **Sistema binario** (base 2): Usa 2 símbolos (0 y 1).
- **Sistema octal** (base 8): Usa 8 símbolos (0-7).
- **Sistema hexadecimal** (base 16): Usa 16 símbolos (0-9 y A-F, donde los valores 10 a 15 se reemplazan por las letras mayúsculas A, B, C, D, E y F).

1. El sistema numérico decimal

El sistema de numeración más utilizado en la vida cotidiana y se basa en la **base 10**. Esto significa que emplea diez símbolos o dígitos diferentes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Cada posición de un número en el sistema decimal tiene un valor asociado que es una potencia de 10, dependiendo de la posición del dígito. Por ejemplo, en el número 345:

- El 5 está en la posición de las unidades ($10^0 = 1$),
- El 4 está en la posición de las decenas ($10^1 = 10$),
- El 3 está en la posición de las centenas ($10^2 = 100$).

Entonces, el número 345 se calcula como:

$$(3 * 100) + (4 * 10) + (5 * 1) = 300 + 40 + 5 = \mathbf{345}$$

El sistema decimal también permite representar fracciones usando el punto decimal. Cada posición a la derecha del punto decimal representa una **potencia negativa de 10**. Por ejemplo, en el número **12.34**:

- El 2 está en las unidades ($10^0 = 1$),
- El 1 está en las decenas ($10^1 = 10$),
- El 3 está en las décimas ($10^{-1} = 0.1$),
- El 4 está en las centésimas ($10^{-2} = 0.01$).

Por lo tanto, 12.34 es:

$$(1 * 10) + (2 * 1) + (3 * 0.1) + (4 * 0.01) = 12.34$$

Este sistema es intuitivo porque utilizamos diez dedos para contar, lo que probablemente influyó en su adopción.

Representación de números enteros negativos

En el sistema decimal, los números enteros negativos se representan anteponiendo el signo negativo (-) al número. Por ejemplo, -25 simplemente significa que estamos hablando del valor negativo de 25.

Ejemplo: El número -198 en decimal

Esto indica que el valor es el negativo del número positivo 198:

$$-198 = -(1 \times 10^2 + 9 \times 10^1 + 8 \times 10^0)$$

$$-198 = -(100 + 90 + 8) = -198$$

Representación de fracciones en el sistema decimal.

Las fracciones (o números decimales) en el sistema decimal se representan usando un **punto decimal** para separar la parte entera de la parte fraccionaria. Los dígitos a la derecha del punto decimal corresponden a **potencias negativas de 10**.

Ejemplo: El número decimal 45.67

$$45,67 = (4 \times 10^1) + (5 \times 10^0) + (6 \times 10^{-1}) + (7 \times 10^{-2})$$

Por lo tanto, el valor del número es 45,67.

2. Sistema binario

El sistema binario es la base 2, lo que significa que solo usa dos símbolos: 0 y 1. Es fundamental en informática porque los ordenadores trabajan con circuitos electrónicos que solo pueden tener dos estados: encendido (1) y apagado (0).

Conversión de decimal a binario:

Para convertir un número decimal en binario, se **divide** el número **entre 2** repetidamente, guardando el **residuo** en cada paso. Al final, los residuos forman el número binario de **abajo hacia arriba**.

Ejemplo: Convertir el número decimal 13 en binario:

$$13 \div 2 = 6 \text{ residuo } 1$$

$$6 \div 2 = 3 \text{ residuo } 0$$

$$3 \div 2 = 1 \text{ residuo } 1$$

$$1 \div 2 = 0 \text{ residuo } 1$$

(El residuo de 1 al dividir entre 2 indica que el número es impar y que el dígito binario en la posición de las unidades será 1.)

Entonces, 13 en binario es 1101.

Conversión de binario a decimal:

1. Escribe el número binario y **asigna una potencia de 2** a cada posición, comenzando desde la derecha con $2^2, 2^1, 2^0$, y así sucesivamente.
2. Multiplica cada dígito del número binario por la potencia de 2 correspondiente.
3. Suma los resultados.

Ejemplo: Convertir el número binario 1101 a decimal.

1. Asignar las potencias de 2 a cada posición, de derecha a izquierda:

$$1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0$$

2. Multiplicamos:

$$1 * 8 + 1 * 4 + 0 * 2 + 1 * 1 = 8 + 4 + 0 + 1$$

3. Sumamos los resultados:

$$8 + 4 + 0 + 1 = 13$$

Por lo tanto, el número binario 1101 en decimal es **13**.

Ejemplo adicional: Convertir el número binario 10110 a decimal.

1. Asignamos las potencias de 2 a cada posición:

$$1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

2. Multiplicamos:

$$1 * 16 + 0 * 8 + 1 * 4 + 1 * 2 + 0 * 1 = 16 + 0 + 4 + 2 + 0$$

3. Sumar:

$$16 + 0 + 4 + 2 + 0 = 22.$$

Por lo tanto, el número binario 10110 en decimal es 22.

Sistema numérico octal (base 8)

El sistema octal es un sistema de numeración en base 8, lo que significa que utiliza **ocho dígitos**: 0, 1, 2, 3, 4, 5, 6, 7. Cada posición de un número octal representa una potencia de 8, comenzando desde la derecha hacia la izquierda. El sistema octal fue ampliamente utilizado en la informática temprana porque se puede relacionar fácilmente con el sistema binario.

Ejemplo: Convertir el número octal 157 a decimal.

$$\begin{aligned} 157_{(8)} &= 1 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 \\ &= 1 \times 64 + 5 \times 8 + 7 \times 1 = 64 + 40 + 7 = 111_{(10)} \end{aligned}$$

Por lo tanto, 157 en octal es equivalente a 111 en decimal.

El octal es útil porque cada dígito octal corresponde a tres bits en binario, lo que facilita las conversiones entre ambos sistemas.

Ejemplo de conversión binario-octal:

El binario 110101 puede agruparse en grupos de tres bits: 110 y 101.

Esto da como resultado el número octal 65.

Fórmula para convertir octal a binario:

- 1. Toma cada dígito del número octal** de izquierda a derecha.
- 2. Convierte cada dígito octal** a su representación binaria de 3 bits.
- 3. Une todos los grupos de 3 bits** para obtener el número binario completo.

Ejemplo de tabla de equivalencias octal-binario:

Octal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Ejemplo: Convertir el número octal 725 a binario.

1. Toma cada dígito del número octal: 7, 2, 5.
2. Convierte cada dígito octal a su representación binaria:
 - 7 en octal = 111 en binario.
 - 2 en octal = 010 en binario.
 - 5 en octal = 101 en binario.
3. Une los grupos de 3 bits:
 - 111 010 101

El número octal 725 es igual a 111010101 en binario.

Sistema numérico hexadecimal (base 16)

El sistema hexadecimal es un sistema de numeración en base 16, que utiliza 16 símbolos: los dígitos del 0 al 9 y las letras A, B, C, D, E, F para representar los valores de 10 a 15. Cada posición en un número hexadecimal representa una potencia de 16.

El hexadecimal es muy utilizado en programación, especialmente para representar direcciones de memoria y valores binarios de una manera más compacta y fácil de leer.

Ejemplo: Convertir el número hexadecimal 2A3 a decimal.

$$2A3_{(16)} = 2 \times 16^2 + A \times 16^1 + 3 \times 16^0$$

Donde A es igual a 10 en decimal:

$$= 2 \times 256 + 10 \times 16 + 3 \times 1 = 512 + 160 + 3 = 675_{(10)}$$

Por lo tanto, 2A3 en hexadecimal es 675 en decimal.

El sistema hexadecimal es útil porque cada dígito hexadecimal se puede representar con cuatro bits en binario, lo que facilita las conversiones entre binario y hexadecimal.

Ejemplo de conversión binario-hexadecimal:

- El binario 11101101 se agrupa en grupos de cuatro bits: 1110 y 1101.
 - Esto da como resultado el número hexadecimal ED.
1. Toma cada dígito hexadecimal de izquierda a derecha.
 2. Convierte cada dígito hexadecimal a su equivalente binario de 4 bits.
 3. Une todos los grupos de 4 bits para obtener el número binario completo.

Tabla de equivalencias hexadecimal-binario:

Hexadecimal	Binario
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Ejemplo: Convertir el número hexadecimal 3F9 a binario.

1. Toma cada dígito hexadecimal: 3, F, 9.
2. Convierte cada dígito a su equivalente binario de 4 bits:
 - 3 en hexadecimal = 0011 en binario.
 - F en hexadecimal = 1111 en binario.
 - 9 en hexadecimal = 1001 en binario.
3. Une los grupos de 4 bits:
 - 0011 1111 1001

El número hexadecimal **3F9** es igual a **001111111001** en binario.